Improving data integrity with DLT

In the last chapter, we introduced DLT as a helpful tool for streaming data and pipeline development. Here, we focus on how to use DLT as your go-to tool for actively tracking data quality. Generally, datasets are dynamic, not neat, and tidy like they often are in school and training. You can use code to clean data, of course, but there is a feature that makes the cleaning process even easier: DLT's expectations. DLT's expectations catch incoming data quality issues and automatically validate that incoming data passes specified rules and quality checks. For example, you might expect your customer data to have positive values for age or that dates follow a specific format. When data does not meet these expectations, it can negatively impact downstream data pipelines. With expectations implemented, you ensure that your pipelines won't suffer.

Implementing expectations gives us more control over data quality, alerting us to unusual data requiring attention and action. There are several options when dealing with erroneous data in DLT:

1. First, we can set a warning, which will report the number of records that failed an expectation as a metric but still write those invalid records to the destination dataset; see `@dlt.expect()` in *Figure 4.1* for an example.
2. Second, we can drop invalid records so that the final dataset only contains records that meet our expectations; see `@dlt.expect_or_drop()` in *Figure 4.1*.
3. Third, we can fail the operation entirely, so nothing new is written (note that this option requires manual re-triggering of the pipeline).
4. Finally, we can quarantine the invalid data to another table to investigate it further. The following code should look familiar to the DLT code in *Chapter 3*, but now with the addition of expectations.

```
def generate_table():
  table_name = spark.conf.get('table_name')
  @dlt.table(name=f'{table_name}',table_properties={"quality":"bronze"})
  @dlt.expect_or_drop("valid_CustomerID", "CustomerID IS NOT NULL")
  @dlt.expect("valid_Product", "Product IS NOT NULL")
  def create_table():
    return build_autoloader_stream()
```

Figure 4.1 – Using DLT expectations to enforce data quality

Let's look at our streaming transactions project as an example. In the *Applying our learning* section in *Chapter 3*, we used DLT to write the transaction data to a table. Utilizing the same DLT code, we will save ourselves the manual effort of cleaning the `CustomerID` column by adding an expectation to the original code to drop any records with a `null` `CustomerID`. We will set another expectation to warn us if the `Product` field is `null`.